

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_3

3. Factory Method Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Factory Method pattern. The Factory Method pattern will make more sense to you if you understand the pros and cons of the Simple Factory pattern, which is covered in Chapter [2](#).

GoF Definition

It defines an interface for creating an object, but lets subclasses decide which class to instantiate. The Factory Method pattern lets a class defer instantiation to subclasses.

Concept

Here you start with an abstract creator class (often called a creator) that defines the basic structure of an application, and the subclasses (that derive from this abstract class) take the responsibility of doing the actual instantiation process. The concept will make sense to you when you analyze the following examples.

Real-Life Example

The example from the Simple Factory pattern also applies here. Let's revisit it with a different perspective: you know that a car manufacturing company produces different models of a car every year. Depending on a market survey, it decides on a model and starts manufacturing. Based on the model of the car, different parts are built and assembled. This company should always be prepared for customers opting for better models in the near future. If the company needs to do a whole new setup for a new model with only a few new features, that can hugely impact the company's profit margin. So, the company should set up the factory in such a way that it can produce parts for the upcoming models easily.

Computer World Example

In database programming, you may need to support different database users. For example, one user may use SQL Server and the other may opt for Oracle. So, when you need to insert data into your database, first you need to create a connection object, such as `SqlServerConnection` or `OracleConnection`, and only then can you proceed. If you put the code into an `if-else` block (or `switch` statements), you may need to repeat lots of similar code, which isn't easily maintainable. Also, whenever you start supporting a new type of connection, you need to reopen your code and make modifications. This type of problem can be resolved using the Factory Method pattern.

- Since the Simple Factory pattern is the simplest form of the Factory Method pattern, let's consider the same examples here. You can say that the static method `getInstance()` of the `java.text.NumberFormat` class is an example of this category. But the following one is a better example.
- The `iterator()` method of the `AbstractCollection<E>` (or `Collection<E>`) is an example of a factory method in Java.

Implementation

In the upcoming example, an abstract creator class called `AnimalFactory` defines the basic structure of the program. As per the definition, the instantiation process is carried out through the subclasses that derive from this abstract class.

I use the same inheritance hierarchy as the Simple Factory pattern in Chapter 2, so this time also you'll see that both the `Dog` and `Tiger` classes implement the `Animal` interface.

The `Animal` hierarchy is the same: there is no change in the `Animal.java`, `Dog.java`, and `Tiger.java`. So, you'll see the following inheritance hierarchy:

```
interface Animal{
    void displayBehavior();
}
class Dog implements Animal
{
    public Dog(){
        System.out.println("\nA dog is created.");
    }
    public void displayBehavior(){
        System.out.println("It says: Bow-Wow.");
        System.out.println("It prefers barking.");
    }
}
```

```

    }
}
class Tiger implements Animal
{
    public Tiger(){
        System.out.println("\nA tiger is created.");
    }
    public void displayBehavior(){
        System.out.println("It says: Halum.");
        System.out.println("It loves to roam in a jungle.");
    }
}
}

```

You'll see another inheritance hierarchy where two concrete classes called `DogFactory` and `TigerFactory` are used to create dog and tiger objects. Each inherits from an abstract class called `AnimalFactory`. These two concrete classes are used to defer the instantiation process. I've attached supportive comments for your easy understanding of this hierarchy.

```

abstract class AnimalFactory{
    // This is the "factory method"
    // Notice that I defer the instantiation process
    // to the subclasses.
    protected abstract Animal createAnimal();
}
// The DogFactory class is used to create dogs
class DogFactory extends AnimalFactory
{
    // Creating and returning a 'Dog' instance
    @Override
    protected Animal createAnimal() {
        return new Dog();
    }
}
// The TigerFactory class is used to create tigers
class TigerFactory extends AnimalFactory
{
    // Creating and returning a 'Tiger' instance
    @Override
    protected Animal createAnimal() {
        return new Tiger();
    }
}
}

```

The remaining code is easy to understand. Let's go to the next section.

Class Diagram

The class diagram in Figure 3-1 shows the key participants of the program.

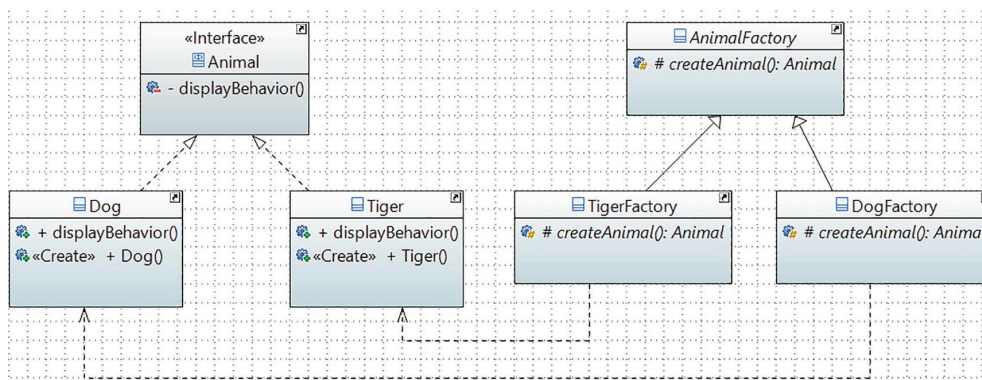


Figure 3-1 Class diagram

Package Explorer View

Figure 3-2 shows the high-level structure of the program. Since this figure cannot be accommodated on the remaining page, let us move to the next page.

factorymethod

implementation_1

Animal.java

Animal

displayBehavior() : void

AnimalFactory.java

AnimalFactory

createAnimal() : Animal

Client.java

Client

main(String[]) : void

Dog.java

Dog

Dog()

displayBehavior() : void

DogFactory.java

DogFactory

createAnimal() : Animal

Tiger.java

Tiger

Tiger()

displayBehavior() : void

TigerFactory.java



Figure 3-2 Package Explorer view

Demonstration 1

Here is the complete demonstration. You can see the complete code inside the package `jdp3e.factorymethod.implementation_1`.

```
// Animal.java
interface Animal{
    void displayBehavior();
}

// Dog.java
class Dog implements Animal{
    public Dog(){
        System.out.println("\nA dog is created.");
    }
    public void displayBehavior(){
        System.out.println("It says: Bow-Wow.");
        System.out.println ("It prefers barking.");
    }
}

// Tiger.java
class Tiger implements Animal{
    public Tiger() {
        System.out.println("\nA tiger is created.");
    }
    public void displayBehavior(){
        System.out.println("Tiger says: Halum.");
        System.out.println("It loves to roam in a jungle.");
    }
}

// AnimalFactory.java
abstract class AnimalFactory{
    // This is the "factory method"
    // Notice that I defer the instantiation
    // process to the subclasses.
    protected abstract Animal createAnimal();
}

// DogFactory.java
class DogFactory extends AnimalFactory
{
    // Creating and returning a 'Dog' instance
    @Override
```

```
        protected Animal createAnimal() {
            return new Dog();
        }
    }
}
// TigerFactory.java
class TigerFactory extends AnimalFactory
{
    // Creating and returning a 'Tiger' instance
    @Override
    protected Animal createAnimal() {
        return new Tiger();
    }
}
// Client.java
class Client {
    public static void main(String[] args) {
        System.out.println("***Factory Method Pattern Demo***");
        AnimalFactory factory;
        Animal animal;
        // Create a tiger and display its behavior
        // using TigerFactory.
        factory =new TigerFactory();
        animal = factory.createAnimal();
        animal.displayBehavior();
        // Create a dog and display its behavior
        // using DogFactory.
        factory =new DogFactory();
        animal = factory.createAnimal();
        animal.displayBehavior();
    }
}
```

Output

Here's the output:

```
***Factory Method Pattern Demo***
A tiger is created.
It says: Halum.
It loves to roam in a jungle.
A dog is created.
It says: Bow-Wow.
It prefers barking.
```

Modified Implementation

You just saw the simplest form of the Factory Method pattern. In demonstration 1, each factory can produce only one type of object, either a dog or a tiger, but there is no variation. For example, it makes sense if a dog factory makes dogs with different colors. Similarly, a tiger factory can produce tigers with different colors. If you receive such a requirement, how can you proceed? The answer is simple: you pass a `color` parameter to the factory method. When you do this, it's called a parameterized factory method.

In addition, notice that the `AnimalFactory` class is an abstract class. **It means that you can take advantage of using an abstract class. How? Ok, let's suppose you want all the subclasses to follow a common rule that is imposed from the parent (or base) class. It simply means that you need to put the common behaviors/codes inside the parent class.** In demonstration 1, you called the `createAnimal()` method for the factory object. This time you do not call this method directly. Instead, you use another method named `createAndDisplayAnimal()` inside `AnimalFactory` to invoke `createAnimal()`. This new method looks like the following (note the changes in bold):

```
abstract class AnimalFactory {
    public void createAndDisplayAnimal(String color){
        Animal animal;
        animal=createAnimal(color);
        animal.displayBehavior();
    }
    // This is the "factory method"
    // Notice that I defer the instantiation
    // process to the subclasses.
    protected abstract Animal createAnimal(String color);
}
```

You can see that when you invoke the `createAndDisplayAnimal()` method, this method invokes the factory method. Once it receives the object from the factory method, you can place the common activities right below this line of code.

Demonstration 2

Now the other parts of the program need to adopt the change. You can see all parts of the program inside the `jdp3e.factorymethod.implementation_2` package on the Apress website. Here is the complete program with important changes in bold:

```
// Animal.java
interface Animal{
    void displayBehavior();
}
```

```

}
// Dog.java
class Dog implements Animal{
    public Dog(String color){
        System.out.println("\nA dog with " + color+ " color is created."
    }
    public void displayBehavior(){
        System.out.println("It says: Bow-Wow.");
        System.out.println ("It prefers barking.");
    }
}
// Tiger.java
class Tiger implements Animal {
    public Tiger(String color){
        System.out.println("\nA tiger with " + color+ " color is created
    }
    public void displayBehavior(){
        System.out.println("It says: Halum.");
        System.out.println("It loves to roam in a jungle.");
    }
}
// AnimalFactory.java
abstract class AnimalFactory {
    public void createAndDisplayAnimal(String color){
        Animal animal;
        animal=createAnimal(color);
        animal.displayBehavior();
    }
    // This is the "factory method"
    // Notice that I defer the instantiation
    // process to the subclasses.
    protected abstract Animal createAnimal(String color);
}
// DogFactory.java
class DogFactory extends AnimalFactory {
    // Creating and returning a 'Dog' instance
    @Override
    protected Animal createAnimal(String color) {
        return new Dog(color);
    }
}
// TigerFactory.java
class TigerFactory extends AnimalFactory {
    // Creating and returning a 'Tiger' instance
    @Override
    protected Animal createAnimal(String color) {
        return new Tiger(color);
    }
}
}

```

```
// Client.java
class Client {
public static void main(String[] args) {
    System.out.println("***Factory Method pattern modified demonstration.*
    AnimalFactory factory;
    // Create a tiger and display its behavior
    // using TigerFactory.
    factory =new TigerFactory();
    factory.createAndDisplayAnimal("yellow");
    // Create a dog and display its behavior
    // using DogFactory.
    factory = new DogFactory();
    factory.createAndDisplayAnimal("white");
}
}
```

Output

Here is the modified output:

```
***Factory Method pattern modified demonstration.***
A tiger with yellow color is created.
It says: Halum.
It loves to roam in a jungle.
A dog with white color is created.
It says: Bow-Wow.
It prefers barking.
```

Analysis

In this example, `animal.displayBehavior()` is the common code that all subclasses use. So, you need to get an animal object before you invoke the `displayBehavior()` method. If you do not depend on this animal instance, you can place any other common code before you invoke the `createAnimal()` method from the `createAndDisplayAnimal()` method. Here is a sample:

```
public void createAndDisplayAnimal(String color){
    System.out.println("Showing common behaviors of an animal.");
    Animal animal;
    animal=createAnimal(color);
    animal.displayBehavior();
}
```

Q&A Session

3.1 Why have you separated the `createAnimal()` method from the client code?

If you read the GoF definition, you understand that this is the main purpose: for the subclasses to create specialized objects. If you look carefully, you will also find that only this “creational part” varies across the products. I discussed this in the Q&A Session of Chapter [2](#).

3.2 What are the advantages of using a factory like this?

Here are some key advantages:

- You are separating the code that varies from the code that does not vary (in other words, the advantages of using the Simple Factory pattern are still present). This helps you maintain the code easily.
- The code is not tightly coupled, so you can add new classes such as `Lion` and `Bear` at any time in the system without modifying the existing architecture. In other words, you have followed the Open/Closed Principle.
- A client can use a factory method without knowing how the actual type of object is created.

3.3 What are the challenges of using a factory like this?

Here are some pitfalls:

- When you use the non-parameterized factory method, you introduce more classes (compared to Simple Factory) in the application. It also indicates that you need to put more effort into unit testing. In addition, notice that each particular instance is associated with a subclass. It means that even if you need only one particular instance, you are forced to create a new subclass. Obviously, the use of a parameterized factory method does not increase the total number of classes and can help you overcome this kind of difficulty.
- If you do not start with a factory method, it can be a challenging task to refactor existing code. You need to understand the dependencies clearly before you refactor the existing code into the factory method pattern.

3.4 I see that the Factory Method pattern is supporting two parallel hierarchies. Is this understanding correct?

Good catch. Yes, from the class diagram, it is evident that this pattern supports parallel class hierarchies; I’ve marked them in Figure [3-3](#).

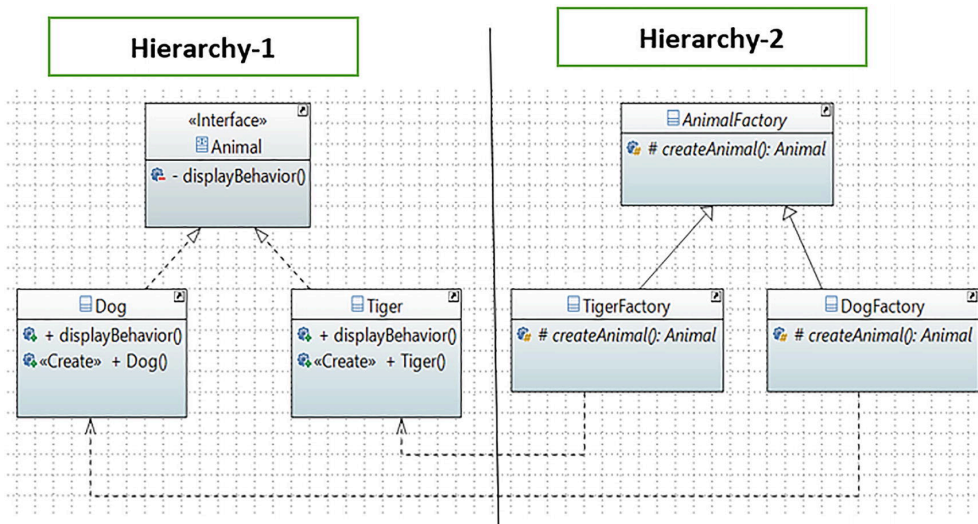


Figure 3-3 The different inheritance hierarchies in this example

In short, `AnimalFactory`, `DogFactory`, and `TigerFactory` are placed in one hierarchy, and `Animal`, `Dog`, and `Tiger` are placed in another hierarchy. Note that the creators and their creations/products form separate hierarchies that run in parallel.

3.5 You should always mark the factory method with an `abstract` keyword so that subclasses can complete them. Is this correct?

No. Sometimes you may be interested in a default factory method if the creator has no subclasses. In that case, you cannot mark the factory method with an `abstract` keyword.

However, to see the real power of the Factory Method pattern, you may need to follow the design that is implemented here in most of the cases.

3.6 It still appears to me that the Factory Method pattern is not that much different from the Simple Factory pattern. Am I correct?

If you look at the subclasses in the examples in both chapters, you may find some similarities. But you should not forget the key aim of the Factory Method pattern; it supplies the framework through which different subclasses can make different products. In the case of the Simple Factory pattern, you cannot similarly vary the products. You can think of the Simple Factory pattern as a one-time deal, but most importantly, your creational part will not be closed for modification. Whenever you want to represent something new, you need to add some code into the `if-else` block or a `switch` statement in the factory class of your Simple Factory pattern.

In this context, always keep in mind the GoF definition, which says “the Factory Method pattern lets a class defer instantiation to subclasses.”

Examine the modified implementation closely. You can see that the `createAnimal()` method is used to create a dog or a tiger using the appropriate subclasses of `AnimalFactory`. So, `createAnimal()` is the factory method that is abstract in this design. When `createAndDisplayAnimal(...)` uses `createAnimal(...)` inside its body, it has no clue whether it is going to work on a dog or a tiger. The subclasses of `AnimalFactory` only know how to create the concrete implementations (a dog or a tiger) for this application.

3.7 I have a doubt: you told me that `getInstance()` of the `java.text.NumberFormat` class is an example of the Factory Method pattern, but I do not see any subclassing involved there. Am I missing something very basic?

Nice observation. You are making good progress. At the end of Chapter 2, you saw that it is a static method that returns the objects. So, technically it is not a perfect example of a Factory Method pattern. Notice that the GoF uses the words “defer instantiation to subclasses,” which tells us that the subclasses should return the actual instances only.

This is why I mention that the `iterator()` method of `AbstractCollection<E>` (or `Collection<E>`) is a better example of the factory method in Java. Let’s retrieve its definition in the Eclipse IDE:

```
public abstract class AbstractCollection<E> implements Collection<E> {
    // Previous code skipped
    public abstract Iterator<E> iterator();
    // Remaining code skipped
}
```

What does it mean? You understand that subclasses of this class need to provide the concrete implementation for the `iterator()` method. Here is such an implementation from the `ArrayList<E>` class:

```
public class ArrayList<E> extends AbstractList<E>
    implements List<E>, RandomAccess, Cloneable,
    java.io.Serializable {
    // Previous code skipped
    public Iterator<E> iterator() {
        return new Itr();
    }
    // Remaining code skipped
}
```

Similarly, `AbstractList<E>` that extends `AbstractCollection<E>` has the following method:

```
public Iterator<E> iterator() {  
    return new Itr();  
}
```

Now analyze the in-built inheritance hierarchy, which is shown in Figure 3-4.



Figure 3-4 Inheritance hierarchy from `AbstractCollection<E>` to `LinkedList<E>`

In short, you can see that the subclasses of `AbstractCollection<E>` such as `ArrayList` and `LinkedList` use the `iterator()` method as per their needs.

You'll learn more about the Iterator pattern in Chapter 18.

3.8 I can say that in the case of the Factory Method pattern, I use a subclassing mechanism (i.e., inheritance) and then implement the factory method that is defined in the parent class. Am I correct?

The answer to this question is yes if you want to strictly follow the GoF definitions. But at the same time, in this context, it is important to note that in many applications/implementations, you may not see the use of an abstract class or interface. For example, in Java, if you can create an XML reader object like this:

```
// Some code before...
XMLReader xmlReader1 = XMLReaderFactory.createXMLReader();
// Some code after
```

This `XMLReaderFactory` is a final class in Java (`XMLReaderFactory` is deprecated since version 9). So, you cannot inherit from it. But see the recommended `SAXParserFactory` class that can be used like the following:

```
// Some code before.
SAXParserFactory factory = SAXParserFactory.newInstance();
SAXParser parser = factory.newSAXParser();
XMLReader xmlReader2 = parser.getXMLReader();
// Some code after.
```

This `SAXParserFactory` is an abstract class.

Previous chapter

< [Part II. The Gang of Four \(GoF\) Design Patterns](#)

Next chapter

[4. Abstract Factory Pattern](#) >